# Feedback Basics: Open and Closed Loop and the Block Diagram

## Contents


1. Overview . . . 2
2. Open-loop control: acting from the plan alone . . . 3
   - 2.1 What sets the robot's speed: push, drag and the ramp's pull . . . 4
   - 2.2 Choosing the push from a test . . . 5
   - 2.3 Where the fixed push breaks: the ramp and the carpet . . . 7
3. Closed-loop control: measure, compare, correct . . . 8
   - 3.1 Measure, compare, correct . . . 8
   - 3.2 The ramp again . . . 9
4. The block diagram . . . 10
   - 4.1 The parts of the loop, named on the robot . . . 10
   - 4.2 One trip round the loop . . . 12
5. Why feedback helps, and what it costs . . . 13
   - 5.1 A disturbance shrinks 11 times . . . 13
   - 5.2 A wrong model: the carpet . . . 15
   - 5.3 What feedback costs . . . 16
6. Holding a value and following a plan: regulation and tracking . . . 17
7. Where the loop runs: Autoware's speed controller . . . 18
8. Summary . . . 20
9. Sources . . . 20
10. Key terms . . . 21

**Where this fits:**

- **Builds on:** State, controls and measurements.
- **Leads to:** Proportional control; Block-diagram algebra; Feedforward plus feedback.

## 1. Overview

**Key point:** A plan says how fast to go, but a fixed push cannot hold that speed when the world differs from the test: a gentle ramp slows the robot from 0.5 to 0.108 m/s. Feedback measures the speed, compares it with the plan and corrects the push, so the same ramp costs only 0.036 m/s.

The robot can now plan its drive: trajectory optimisation gives it the speeds and turns that take it past the table to the door, within its limits. A plan is only a list of wanted values over time, such as "drive at 0.5 m/s now". Something must turn each wanted value into a push from the motors, and keep the robot on it when the world does not match the plan.

Take the simplest part of that job: holding 0.5 m/s along a corridor. A test on the flat hard floor shows that a steady push of 1 N from the motors holds 0.5 m/s. Figure 1 plays two copies of the robot. Both cruise at 0.5 m/s and, after 4 s, reach a gentle ramp that rises 1 cm for every metre along it. Watch their speeds in the lower panel:

- the top copy keeps pushing the 1 N of the test, and the ramp slows it to 0.108 m/s;
- the bottom copy measures its speed 100 times a second and pushes harder when it is too slow; it settles at 0.464 m/s.

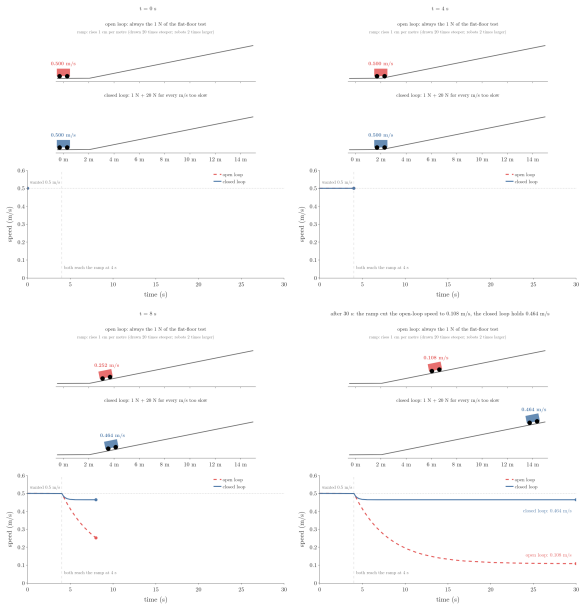


**Figure 1:** Two copies of the robot, seen from the side, cruise at 0.5 m/s and reach a ramp that rises 1 cm per metre at x = 2 m, t = 4 s; one frame per second for 30 s, no sensor noise. Top: open loop, always the 1 N of the flat-floor test. Middle: closed loop, 1 N plus 20 N for every m/s too slow. Bottom: their speeds over time. The slope is drawn 20 times steeper and the robots 2 times larger than they are. The last frame: open loop 0.108 m/s, closed loop 0.464 m/s

A carpet does the same kind of damage: with the same 1 N, carpet's stronger drag gives only 0.333 m/s (Section 2.3). The fixed push fails because nothing looks at the speed; the second copy looks and corrects. This Note builds that idea:

- acting from the plan alone, and why the ramp and the carpet break it (Section 2);
- measuring, comparing and correcting (Section 3);
- the block diagram, the standard picture and names for every part of the loop (Section 4);
- why feedback helps, and what it costs (Section 5);
- holding a fixed value against following a changing one (Section 6);
- where this loop runs in real vehicles (Section 7).

## 2. Open-loop control: acting from the plan alone

> **Key point:** Without measuring, the only way to hold 0.5 m/s is to know in advance which push gives 0.5 m/s. A test on the flat floor finds 1 N, but the same 1 N gives 0.108 m/s on the ramp and 0.333 m/s on carpet, because the push was worked out for a floor that has changed.

### 2.1 What sets the robot's speed: push, drag and the ramp's pull

To see why the top copy of Figure 1 slows on the ramp, we first need what sets the robot's speed. Three forces act along its direction of travel:

- the motors push it forward;
- the floor's rolling drag holds it back, more the faster it rolls;
- on a ramp, part of its weight pulls it back down the slope.

Whatever is left over speeds the robot up or slows it down, by **Newton's second law** (G-2809): the net force on a body equals its mass times its acceleration (MR §8.1.1). For our robot:

- $m$ is its mass, 8 kg;
- $v$ is its speed along the corridor, in m/s, 0.5 m/s when cruising;
- $u$ is the push of the motors, in N, between $-4$ and 4 N; it is the robot's **control input** (G-2298), the number we choose;
- $b$ is the drag coefficient, 2 N per m/s on the hard floor, so the drag at 0.5 m/s is 1 N;
- $d$ is the ramp's pull, in N, 0 on the flat;
- $\dot{v}$ is the rate of change of the speed, the acceleration, in m/s$^2$ (the dot is the **dot notation** (G-2296) for "rate of change per second").

$$m\,\dot{v} = u - b\,v - d$$

**The ramp's pull.** On a slope, the part of the weight that acts along the slope is the weight times the sine of the slope's angle (OpenStax §5.6). The sine is the rise divided by the length along the slope, here 1 cm per metre, 0.01. With gravity's pull of 9.81 N on each kilogram:

$$d = 8 \times 9.81 \times 0.01$$

$$= 0.785 \text{ N}$$

**Check at easy values.** Cruising on the flat with the 1 N of the test:

$$8\,\dot{v} = 1 - 2 \times 0.5 - 0$$

$$= 0$$

Nothing is left over, so the speed stays at 0.5 m/s. On the ramp, with the same push:

$$8\,\dot{v} = 1 - 1 - 0.785$$

$$\dot{v} = -0.785/8 = -0.098 \text{ m/s}^2$$

The robot slows down. The equation also gives the acceleration limit that planning with motion limits uses. From rest there is no drag, and the largest push is 4 N:

$$\dot{v} = 4/8 = 0.5 \text{ m/s}^2$$

The speed is the **state** (G-2314) of this problem, what we must keep track of because it decides what happens next. The equation gives how fast the state changes from the state and the input, which is what a state-space model is: here "rate of change of $v$ = a function of $v$ and $u$". The notebook steps it forward every 10 ms with Euler steps (G-2299). Next: which push holds 0.5 m/s.

### 2.2 Choosing the push from a test

To hold 0.5 m/s without measuring anything, the robot must know in advance which push gives 0.5 m/s. People do the same every day: a toaster's dial is set from experience of how dark the toast came out before, and a shower handle is turned to where the water was warm yesterday. Neither looks at the result while it works.

For the robot, the experience is a test on the hard floor. A steady push of 0.5 N settles at 0.25 m/s. Once the speed has settled it no longer changes, so its rate of change is 0 and the push exactly balances the drag:

$$0 = 0.5 - 2\,v$$

$$v = 0.5/2 = 0.25 \text{ m/s}$$

The same steps in symbols, on a flat floor, with the push $u$, the drag coefficient $b$ and the speed $v$ of Section 2.1:

$$0 = u - b\,v$$

$$v = u/b$$

The settled speed grows in step with the push, so doubling the push to 1 N doubles the speed to 0.5 m/s. In symbols, the push for a wanted speed is the drag coefficient times that speed:

$$u = b\,v = 2 \times 0.5 = 1 \text{ N}$$

Choosing the commands from the plan and a model alone, with no measurement of what the robot does, is **open-loop control** (G-2940) (FBS §1.1; MR §11.3). Figure 2 (top) draws it as a chain: plan, computer, motors, robot, and nothing comes back.

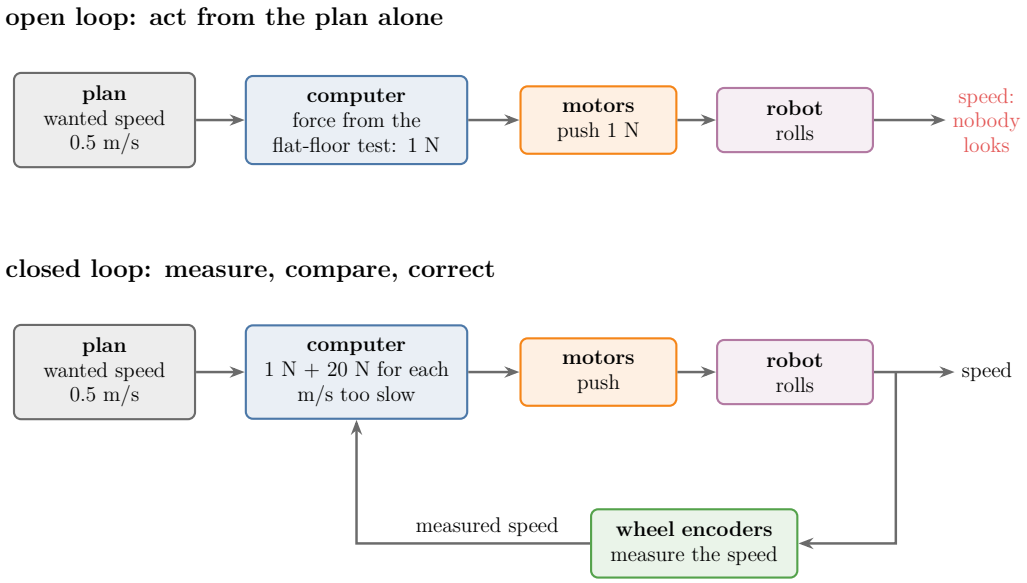



**Figure 2:** Open loop (top): the plan's wanted speed, 0.5 m/s, goes to the computer, which sends the 1 N found in the flat-floor test to the motors; nobody looks at the speed. Closed loop (bottom): the wheel encoders measure the speed and send it back to the computer, which pushes 1 N plus 20 N for each m/s the robot is too slow

Figure 3 shows the test and what the same 1 N does elsewhere. Each line gives the speed a constant push settles at: blue on the hard floor, where the test was made, red dashed on the ramp, purple dotted on carpet. Read up from 1 N on the horizontal axis.

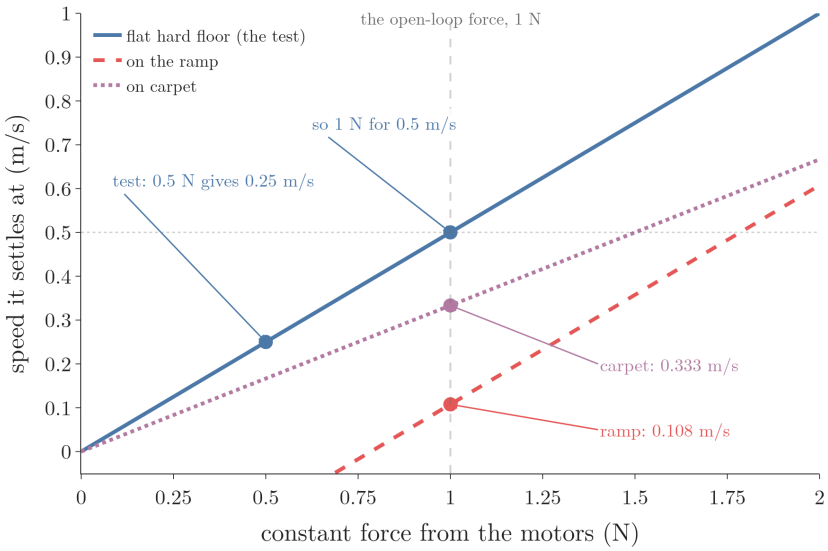



**Figure 3:** The speed a constant push settles at, for pushes from 0 to 2 N. Blue: the flat hard floor, drag 2 N per m/s; the test, 0.5 N gives 0.25 m/s, so 1 N is chosen for 0.5 m/s. Red dashed: on the ramp, whose pull of 0.785 N comes off the push; 1 N gives 0.108 m/s. Purple dotted: on carpet, drag 3 N per m/s; 1 N gives 0.333 m/s

The open loop holds 0.5 m/s exactly, but only on the blue line.

### 2.3 Where the fixed push breaks: the ramp and the carpet

The open-loop push of 1 N is right only for the floor it was tested on. On the ramp of Figure 1, the speed settles where the push balances drag and pull together:

$$0 = 1 - 2\,v - 0.785$$

$$2\,v = 0.215$$

$$v = 0.108 \text{ m/s}$$

On carpet the drag is stronger, 3 N per m/s, and there is no ramp:

$$0 = 1 - 3\,v$$

$$v = 0.333 \text{ m/s}$$

Both come from the same balance, with the push $u$, the ramp's pull $d$ and the true drag $b$:

$$v = \frac{u - d}{b}$$

In both cases the motors keep pushing 1 N while the robot crawls, because nothing in the open loop knows the speed (Figure 2, top). A shower behaves the same way when someone starts the dishwasher: the water runs cold, and a handle set from yesterday's experience does not move. To notice, the robot must measure its speed.

## 3. Closed-loop control: measure, compare, correct

> **Key point:** The robot measures its speed, subtracts it from the wanted speed and adds 20 N for every m/s it is too slow. On the ramp this holds 0.464 m/s instead of 0.108 m/s.

### 3.1 Measure, compare, correct

The open loop of Section 2 failed because nothing looked at the speed. In the shower, we feel the water, notice it is too cold and turn the handle further; the colder it is, the further we turn. The robot can do the same:

1. **Measure.** The wheel encoders count how far each wheel turns, so every 10 ms the computer knows the speed.
2. **Compare.** The computer subtracts the measured speed from the wanted speed. The gap, wanted minus measured, is the **control error** (G-2943), or simply the error: how far the robot is from where it should be, with its sign (FBS §1.6).
3. **Correct.** The computer keeps the open-loop push of 1 N and adds 20 N for every m/s of error.

Using a measurement of what the system does to decide its next command is **feedback** (G-2941). Control that uses feedback is **closed-loop control** (G-2942): the measurement runs back from the robot to the computer, closing a loop (FBS §1.1). Figure 2 (bottom) draws it; the only new part is the arrow coming back. The rule, with:

- $r$ is the wanted speed, 0.5 m/s;
- $y_m$ is the measured speed, in m/s;
- $e$ is the control error, in m/s;
- $u_0$ is the open-loop push for the wanted speed, the drag coefficient $b$ times $r$, 1 N;
- $K_p$ is the push added per m/s of error, 20 N per m/s;
- $u$ is the push the computer asks the motors for, in N.

$$e = r - y_m$$

$$u = u_0 + K_p\, e$$

At 0.5 m/s the error is 0 and the push is the 1 N of the test. At 0.45 m/s:

$$e = 0.5 - 0.45 = 0.05 \text{ m/s}$$

$$u = 1 + 20 \times 0.05 = 2 \text{ N}$$

The value 20 N per m/s is our choice. A larger value corrects harder; Section 5 shows what it gains and what it costs.

### 3.2 The ramp again

Does the correction fix the ramp of Figure 1? On the ramp the speed settles where the push balances drag and pull. The push now depends on the speed, through the error, so the speed appears on both sides; numbers first:

$$0 = 1 + 20\,(0.5 - v) - 2\,v - 0.785$$

$$0 = 1 + 10 - 20\,v - 2\,v - 0.785$$

$$22\,v = 10.215$$

$$v = 0.464 \text{ m/s}$$

The same steps in symbols, with the ramp's pull $d$ and the drag $b$ of Section 2.1, and the wanted speed $r$, the open-loop push $u_0$ and the push per m/s of error $K_p$ of Section 3.1:

$$0 = u_0 + K_p\,(r - v) - b\,v - d$$

$$(b + K_p)\,v = u_0 + K_p\,r - d$$

$$v = \frac{u_0 + K_p\,r - d}{b + K_p}$$

The closed loop loses 0.036 m/s on the ramp (0.5 m/s wanted, 0.464 m/s held), against 0.392 m/s for the open loop. Figure 1 shows the same run in time: the closed loop's speed dips within about a second and then holds.

A small error remains because the extra push has to come from somewhere. Holding 0.464 m/s on the ramp needs 0.714 N more than the test's 1 N, and the rule gives extra push only when there is an error:

$$20 \times 0.0357 = 0.714 \text{ N}$$

Feedback has cut the ramp's damage more than tenfold. Its parts now have standard names and a standard picture, in which this controller, and every other one, can be drawn the same way.

## 4. The block diagram

> **Key point:** Every feedback loop has the same parts: a reference, a controller, an actuator, a plant, a sensor, and two unwanted inputs, a disturbance that changes what the plant does and sensor noise that changes only what we measure. A block diagram draws each part as a box and each signal as an arrow.

### 4.1 The parts of the loop, named on the robot

Section 3's loop has four parts, the computer, the motors, the robot and the encoders, and several signals between them. Figure 4 draws them in the standard way. Each part is a box with its job written in it, each signal is an arrow from the part that makes it to the part that uses it, and each circle adds the signals that arrive at it, with the sign written beside each arrow (FBS §2.3, Fig. 2.7). Such a picture is a **block diagram** (G-2950), and a circle that adds and subtracts signals is a **summing junction** (G-2951). A driver holding the speed limit has the same parts, so each one below has its driving example beside the robot's.

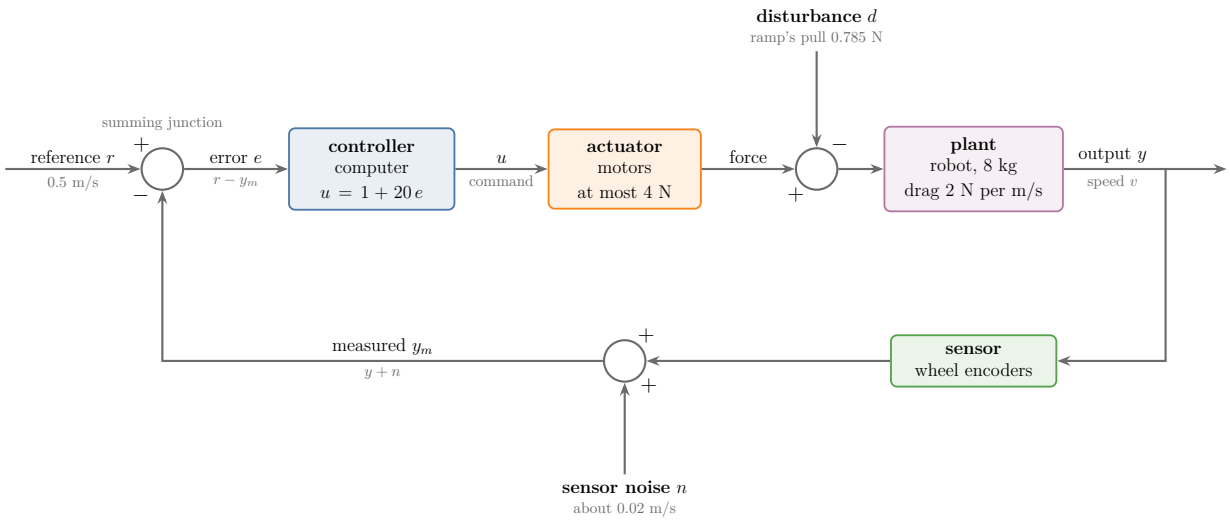



**Figure 4:** The block diagram of the robot's speed loop. The reference r, 0.5 m/s, enters a summing junction, where the measured speed y_m is subtracted to give the error e. The controller, the computer running u = 1 + 20 e, sends the command u to the actuator, the motors, at most 4 N. Their force enters a second summing junction, where the disturbance d, the ramp's pull of 0.785 N, is subtracted. The rest drives the plant, the robot of 8 kg with drag 2 N per m/s, whose output y is the speed v. The sensor, the wheel encoders, measures it; sensor noise n, about 0.02 m/s, is added to the reading, which goes back to the first summing junction as y_m

Follow Figure 4 from the left:

- The **reference** (G-2944) $r$ is the value we want the output to have: 0.5 m/s from the plan; for a driver, the speed limit.
- The **controller** (G-2947) is the part that turns the error into a command: the robot's computer running the rule of Section 3.1, 1 N plus 20 N per m/s of error; for a driver, the brain, and in cruise control, a computer in the car.
- The **actuator** (G-2948) is the part that turns the command into a physical push on the plant: the motor drivers and motors, which can give at most 4 N; for a driver, the foot on the pedal.
- The **plant** (G-2945) is the thing being controlled: the robot's body, rolling with its 8 kg and its drag. Its output $y$, the quantity we control, is the speed $v$; for a driver, the car and its speed.
- The **sensor** (G-2946) is the part that measures the output: the wheel encoders; for a driver, the speedometer.
- The **control error** (G-2943) $e$ is the reference minus the measured output, formed at the first summing junction.

Two more arrows come from outside the loop, and they enter at different places:

- A **disturbance** (G-2949) $d$ is an unwanted outside effect that changes what the plant does: the ramp pulls the real robot back by 0.785 N. Gusts, bumps and a heavier load

are others; for a driver, a hill or ice.

- **Sensor noise** (G-2312) $n$ is the small random error in each reading: the encoders' speed jitters by about 0.02 m/s from reading to reading; for a driver, a blurry speedometer.

A disturbance is not noise. The ramp really slows the robot: its true speed drops. Noise leaves the true speed alone and only makes the reading wrong: the robot rolls at 0.5 m/s while one reading says 0.52 m/s and the next 0.49 m/s. So Figure 4 adds $d$ before the plant and $n$ after it, at the sensor.

The actuator, the plant and the sensor are often drawn as one box and together called the plant or the process (FBS §1.3, Fig. 1.4). Figure 4 names every part, but it is drawn still; next we follow the signals round it.

### 4.2 One trip round the loop

The block diagram is not a still picture: the computer runs once round it every 10 ms, 100 times a second. Figure 5 follows the loop's first cycle on the ramp, at t = 4.00 s, stage by stage; the box or arrow in use is drawn thick, with its number beside it. In this run the sensor noise is 0.

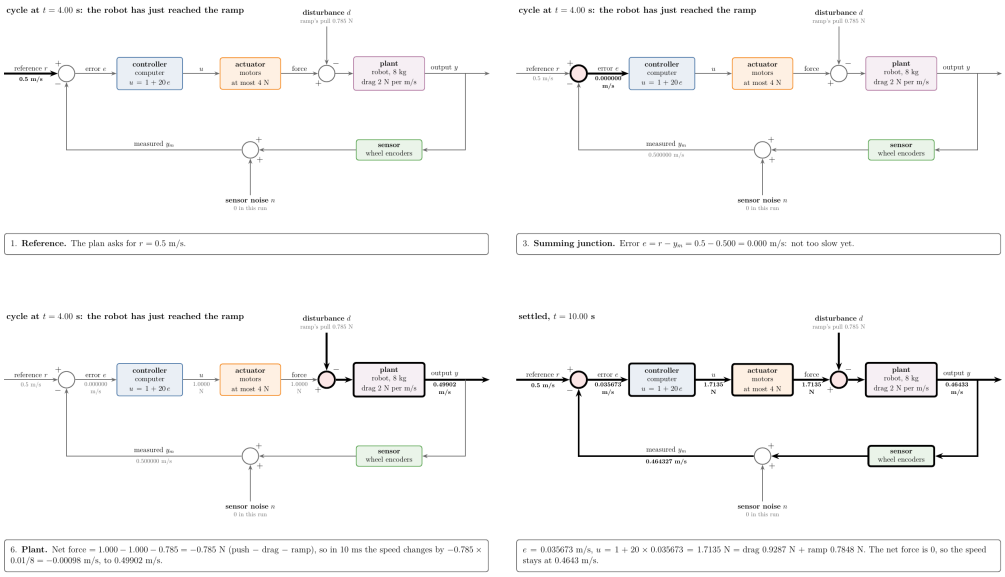



**Figure 5:** The speed loop's block diagram, one stage per frame, with the closed-loop ramp run's numbers and no sensor noise. Cycle at t = 4.00 s: 1 the reference is 0.5 m/s; 2 the sensor reads 0.500 m/s; 3 the error is 0; 4 the controller asks for 1 N; 5 the motors push 1 N; 6 net force 1.000 − 1.000 − 0.785 = −0.785 N, so in 10 ms the speed falls by 0.00098 m/s; 7 the new speed goes back to the sensor. Then the next cycle (error 0.000981 m/s, push 1.0196 N), the loop at t = 4.50 s (error 0.026825 m/s, push 1.5365 N) and settled at t = 10.00 s (error 0.035673 m/s, push 1.7135 N, net force 0)

The cycle at t = 4.00 s, with the plant's equation of Section 2.1 for stage 6:

$$e = 0.5 - 0.500 = 0 \text{ m/s}$$

$$u = 1 + 20 \times 0 = 1 \text{ N}$$

$$\text{net force} = 1 - 1 - 0.785$$

$$= -0.785 \text{ N}$$

$$\text{speed change} = -0.785 \times 0.01/8$$

$$= -0.00098 \text{ m/s}$$

The ramp has not yet registered: the controller still pushes the flat-floor 1 N, and the robot loses a little speed. In the next cycle the error is 0.000981 m/s and the push 1.0196 N, a correction too small to see. Each cycle the error grows a little, the push with it, and the speed falls more slowly. Frame "settled, t = 10.00 s" shows where it stops: an error of 0.0357 m/s asks for 1.7135 N, which is the drag at that speed plus the ramp's pull, so the net force is 0 and the speed stays at 0.464 m/s.

The diagram shows where the ramp enters, at the plant, and where the answer to it comes from, the error. Section 5 uses it to say why the answer works so well.

## 5. Why feedback helps, and what it costs

> **Key point:** Feedback shrinks the speed lost to a disturbance or to a wrong model, 11 times on the ramp and 7.7 times on carpet, because every m/s lost brings 20 N more push. The price: it acts only once an error exists, and it passes the sensor's noise to the motors.

### 5.1 A disturbance shrinks 11 times

On the ramp, the open loop lost 0.392 m/s and the closed loop 0.036 m/s. The ratio is not luck. In the open loop, only the drag answers the ramp: each m/s lost removes 2 N of drag. In the closed loop the controller answers too: each m/s lost also adds 20 N of push. So the same pull costs 11 times less speed. Numbers first:

$$\text{open loop: } 0.785/2 = 0.392 \text{ m/s}$$

$$\text{closed loop: } 0.785/22 = 0.0357 \text{ m/s}$$

$$0.392/0.0357 = 11$$

The same steps in symbols, with the ramp's pull $d$, the drag $b$ and the push per m/s of error $K_p$; the speed lost is the wanted 0.5 m/s minus the settled speed of Section 2.3 and Section 3.2:

$$\text{open loop: } d/b$$

$$\text{closed loop: } d/(b + K_p)$$

$$\text{ratio: } (b + K_p)/b$$

$$= 1 + K_p/b$$

FBS §2.1 states the same rule for any loop: feedback divides the effect of a disturbance on the plant by 1 plus the loop's gain, the push added per m/s of error divided by the drag's own N per m/s:

$$1 + 20/2 = 11$$

The left panel of Figure 6 sweeps the ramp's pull from a 2 percent slope down to a 2 percent slope up: the open-loop speed (red) follows every change, the closed-loop speed (blue) barely moves.

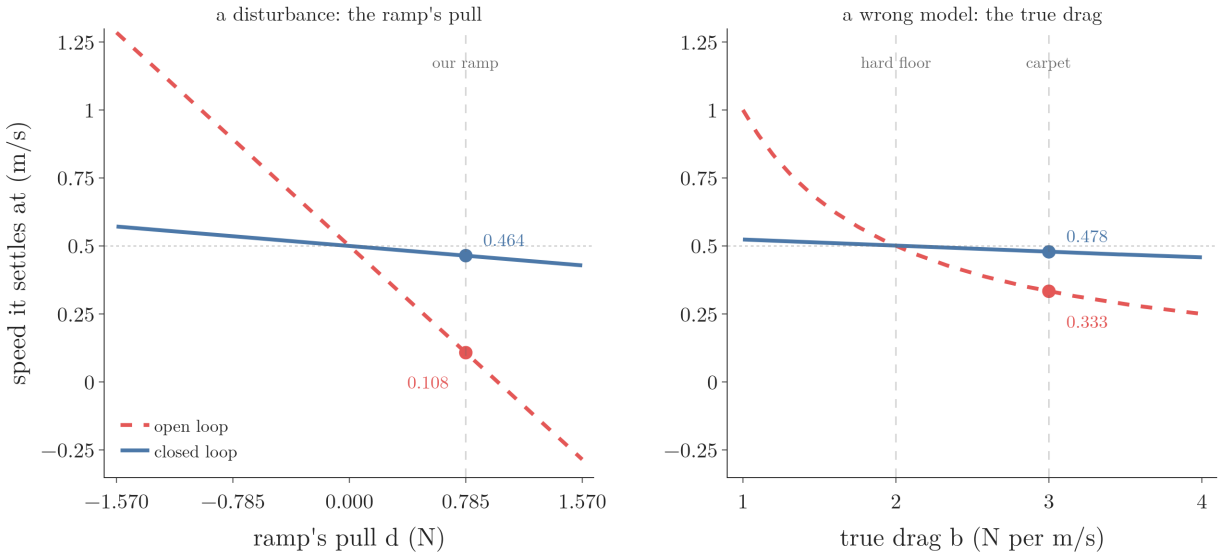



**Figure 6:** The speed each loop settles at. Left: against the ramp's pull d, from −1.570 N (a 2 percent slope down) to +1.570 N (2 percent up); our ramp, 0.785 N, gives 0.108 m/s open loop (red) and 0.464 m/s closed loop (blue). Right: against the true drag b, while the controller still assumes 2 N per m/s; carpet, 3 N per m/s, gives 0.333 m/s open loop and 0.478 m/s closed loop

### 5.2 A wrong model: the carpet

The disturbance was an outside force. The carpet is a different failure: the controller's model is wrong, since its push of 1 N assumes the hard floor's drag. The open loop settles at 0.333 m/s (Section 2.3). The closed loop, with the settled-speed formula of Section 3.2, no ramp and the true drag of 3 N per m/s:

$$v = \frac{1 + 20 \times 0.5}{3 + 20}$$

$$= 11/23 = 0.478 \text{ m/s}$$

The closed loop loses 0.0217 m/s where the open loop loses 0.1667 m/s, 7.7 times less. The rule of Section 5.1 holds again, with carpet's drag of 3 N per m/s in place of 2:

$$0.1667/0.0217 = 7.7$$

$$1 + 20/3 = 7.7$$

Figure 6 (right) sweeps the true drag from 1 to 4 N per m/s: the open loop's speed swings from 1.0 to 0.25 m/s, the closed loop's stays between 0.524 and 0.458 m/s. Feedback works

whatever the cause of the drop, because it reacts to the measured speed, not to a model of why the speed dropped (FBS §1.5). Coping with a model that is wrong is called robustness, the subject of sensitivity and robustness.

### 5.3 What feedback costs

Feedback is not free. Three costs follow from how it works:

1. **It waits for an error.** The correction starts only once the speed has dropped, so the closed loop of Figure 1 still dips, and a small error stays: the 0.036 m/s that pays for the extra 0.714 N (Section 3.2). "Feedback is reactive: there must be an error before corrective actions are taken" (FBS §1.2).
2. **It feeds sensor noise to the motors.** The controller cannot tell noise from a real drop in speed. Figure 7 runs the closed loop on the ramp with readings that jitter by 0.02 m/s (one standard deviation): the push jitters by 20 N per m/s times 0.02 m/s, 0.40 N, while the robot's true speed jitters by only 0.002 m/s, because its 8 kg cannot follow such fast changes. A larger $K_p$, the push per m/s of error, would cut the ramp's error further but shake the motors harder.
3. **Too strong or too late a correction makes a system swing.** Feedback carries a "risk for instability" that a fixed push does not (FBS §1.2, Table 1.1). The speed loop does not swing, but PD and PID control shows the same rule making the robot's turning swing back and forth.

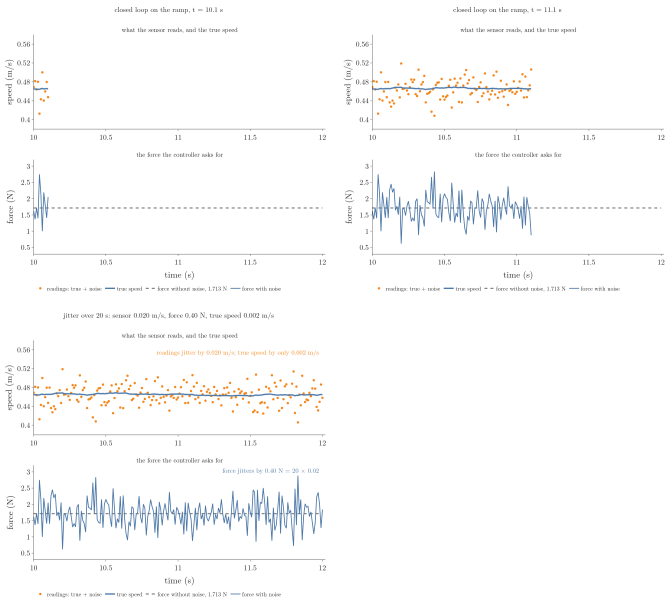



**Figure 7:** The closed loop on the ramp with sensor noise: each 10 ms reading is the true speed plus a random error of typical size 0.02 m/s. From t = 10 to 12 s, ten readings per frame. Top: the readings (orange dots) and the true speed (blue). Bottom: the push the controller asks for (blue) against 1.713 N, its value without noise (grey). The last frame gives the jitter over 20 s: readings 0.020 m/s, push 0.40 N, true speed 0.002 m/s

So far the wanted speed never changed. A plan changes it all the time.

## 6. Holding a value and following a plan: regulation and tracking

> **Key point:** Holding a fixed reference against disturbances is regulation; following a reference that changes is tracking. The same loop does both, but while the plan speeds up at 0.25 m/s$^2$ it lags by 0.091 m/s, because the extra push for speeding up comes only from an error.

So far the reference stayed at 0.5 m/s, and the loop held it against the ramp. Keeping an output close to a fixed reference in spite of disturbances is **regulation** (G-2952) (FBS §2.3), and the fixed reference is a **setpoint** (G-2953) (FBS §11.1; MR §11.3). A plan instead changes the reference over time: from rest, it speeds the robot up at 0.25 m/s$^2$ for 2 s, to 0.5 m/s. Making an output follow a reference that changes is **tracking** (G-2954), also called the servo problem (FBS §2.4); MR §11.3 calls it trajectory tracking. A docked robot holding its place is regulating; a robot driving the plan of trajectory optimisation is tracking.

The loop is the same as in Figure 4, with the open-loop push now following the reference: $b$ times $r$, which grows from 0 to 1 N as the plan speeds up. Figure 8 runs it on this plan. Watch the orange bar, the error, in the lower panel: while the reference rises it grows to 0.091 m/s; once the reference is held, it shrinks to 0.

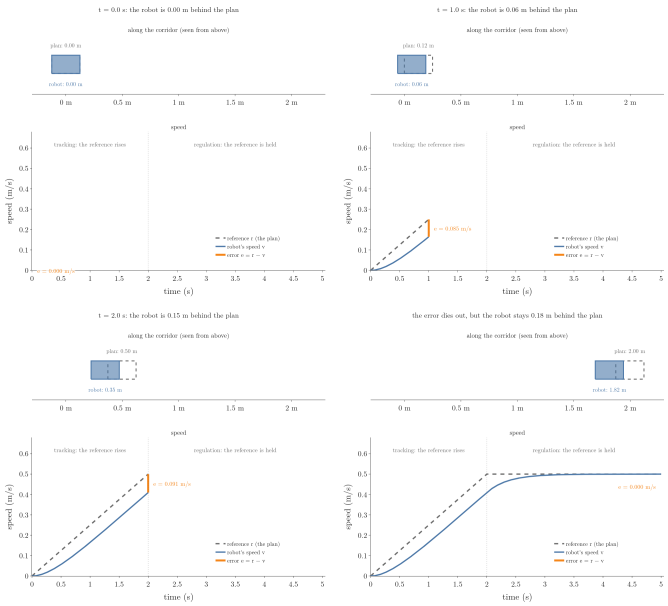


**Figure 8:** Regulation and tracking on one run. The plan speeds the robot up from rest at 0.25 m/s$^2$ for 2 s to 0.5 m/s, then holds it; one frame per 0.2 s for 5 s. Top: the plan's position (grey dashed outline) and the robot (blue) along the corridor. Bottom: the reference speed (grey dashed), the robot's speed (blue) and the error between them (orange). While the reference rises (tracking) the error grows to 0.091 m/s; once it is held (regulation) the error dies out, but the robot stays 0.18 m behind the plan

**Why it lags.** The open-loop push $b$ times $r$ covers the drag at the planned speed, but speeding up 8 kg at 0.25 m/s$^2$ takes another 2 N. The rule gives extra push only through the error: 20 N for each m/s of error, plus 2 N of drag saved for each m/s the robot is slower than the plan. The error grows until the two supply the 2 N; numbers first:

$$8 \times 0.25 = 20\,e + 2\,e$$

$$2 = 22\,e$$

$$e = 2/22 = 0.091 \text{ m/s}$$

The same steps in symbols, with the mass $m$, 8 kg, the drag $b$, 2 N per m/s, the push per m/s of error $K_p$, 20 N per m/s, and:

- $a$ is the plan's acceleration, in m/s$^2$, 0.25 m/s$^2$ while the plan speeds up.

$$m\,a = K_p\,e + b\,e$$

$$e = \frac{m\,a}{K_p + b}$$

MR §11.3 finds the same for a joint following a reference moving at constant speed: with a correction proportional to the error alone, the joint "lags behind" by an amount that shrinks as the gain grows. Once the reference is held at 0.5 m/s, the problem is regulation again and the error dies out. The robot does not catch up, though: it ends 0.18 m behind the plan, because this loop controls the speed, and nothing in it measures the position. Following a planned path in position is the job of the path-tracking chapter.

## 7. Where the loop runs: Autoware's speed controller

The loop of Figure 4 is how real vehicles hold the speeds of a planned trajectory. Autoware, an open-source software stack for self-driving cars, does it in its longitudinal controller, `control/autoware_pid_longitudinal_controller` in the `autoware_universe` repository (read for this Note). Each part of Figure 4 appears in its code:

- **Reference:** the target speed of the trajectory point the car is at, `longitudinal_velocity_mps`, which changes along the trajectory, so the controller is tracking.
- **Sensor:** the measured speed, `current_motion.vel`, from the car's odometry.
- **Control error:** `diff_vel = (target_motion.vel - current_vel)`, in `applyVelocityFeedback` (`src/pid_longitudinal_controller.cpp`).

- **Controller:** a correction computed from the error by `m_pid_vel.calculate(...)`, whose gains are set in `config/autoware_pid_longitudinal_controller.param.yaml` (`kp: 1.0`, `ki: 0.1`, `kd: 0.0`); PD and PID control explains the three. Its output is a target acceleration rather than a force.
- **Actuator:** the vehicle interface, which turns the target acceleration into accelerator and brake commands (README: "the target acceleration calculated here will be properly realized by the vehicle interface").
- **Disturbance:** the road's slope. Autoware also estimates the slope from the car's pitch and adds the gravity term in advance, `9.81 * std::sin(pitch_limited)` in `applySlopeCompensation`: the ramp's pull per kilogram, as in Section 2.1, where it is 0.098 m/s$^2$ for our ramp. Adding a known effect before any error appears is feedforward, the subject of feedforward and cascades.

ROS 2 robots use the same structure through `ros2_control`: its `pid_controller` (in the `ros2_controllers` repository, `pid_controller/doc/userdoc.rst`) takes the reference on a topic, reads the measured value from the hardware's state interfaces, the sensor, and writes its output to a command interface, the actuator.

## 8. Summary

| | Open loop | Closed loop |
|---|---|---|
| Uses | the plan and a test: 1 N for 0.5 m/s | the plan, the test and the measured speed: 1 N + 20 N per m/s of error |
| On the ramp | 0.108 m/s | 0.464 m/s |
| On carpet | 0.333 m/s | 0.478 m/s |
| Costs | nothing measured, nothing corrected | waits for an error, passes sensor noise to the motors |

- The opening question has its answer: on the ramp, the fixed 1 N of **open-loop control** (G-2940) slows the robot from 0.5 to 0.108 m/s, while **closed-loop control** (G-2942) holds 0.464 m/s, because it measures the speed and pushes 20 N harder for every m/s it is too slow.
- Open-loop control chooses the push from a test on the hard floor, so it is right only where the test was made: the ramp's pull and carpet's drag both change the speed the same push gives.
- **Feedback** (G-2941) corrects the push by the **control error** (G-2943), wanted minus measured speed, so the robot reacts to what really happens, whatever the cause.
- A **block diagram** (G-2950) names the parts of every loop, the **reference** (G-2944), **controller** (G-2947), **actuator** (G-2948), **plant** (G-2945) and **sensor** (G-2946), so any controller can be drawn and compared the same way.
- A **disturbance** (G-2949) changes what the plant really does and enters before it; **sensor noise** (G-2312) changes only the reading and enters at the sensor, so the two must not be confused.
- Feedback divides the speed lost to a disturbance or to a wrong model by 1 plus the loop's gain, 11 on the ramp and 7.7 on carpet, because each m/s lost brings 20 N more push besides the drag's own change.
- Feedback costs something: it acts only once an error exists, so a small error of 0.036 m/s stays on the ramp, and it passes sensor noise to the motors, 0.40 N of jitter from 0.02 m/s of noise.
- **Regulation** (G-2952) holds a **setpoint** (G-2953) against disturbances, while **tracking** (G-2954) follows a changing reference; a correction proportional to the error tracks a speeding-up plan only with a lag, here 0.091 m/s, because the push for speeding up comes from the error.

The robot can now hold a speed against ramps and carpets. Still missing: the correction leaves 0.036 m/s on the ramp and lags a changing plan, and the same rule applied to turning makes the robot swing from 0° to 90° and back when it should turn to 45°; PD and PID control fixes both.

## 9. Sources

### Built from

- MATLAB, “Understanding Control Systems, Part 1: Open-Loop Control Systems”, MathWorks video, 2016, https://www.mathworks.com/videos/understanding-control-systems-part-1-open-loop-control-systems-123419.html
- MATLAB, “Understanding Control Systems, Part 2: Feedback Control Systems”, MathWorks video, 2016, https://www.mathworks.com/videos/understanding-control-systems-part-2-feedback-control-systems-123501.html
- MATLAB, “Understanding Control Systems, Part 3: Components of a Feedback Control System”, MathWorks video, 2016, https://www.mathworks.com/videos/understanding-control-systems-part-3-components-of-a-feedback-control-system-123645.html
- MATLAB, “Understanding Control Systems, Part 4: Simulating Disturbance Rejection in Simulink”, MathWorks video, https://www.mathworks.com/videos/understanding-control-systems–part-4–simulating-disturbance-re-1480629735127.html
- MATLAB, “Understanding Control Systems, Part 5: Simulating Robustness to System Variations in Simulink”, MathWorks video, https://www.mathworks.com/videos/understanding-control-systems–part-5–simulating-robustness-to–1481654824778.html
- Northwestern Robotics, “Modern Robotics, Chapter 11.3: Motion Control with Velocity Inputs (Part 1 of 3)”, YouTube, https://www.youtube.com/watch?v=QFCbTVJqm8I
- Åström, K. J. and Murray, R. M. (2021). *Feedback Systems*, 2nd ed., §1.1 “What Is Feedback?”, §1.2 “What is Feedforward?”, §1.5 “Feedback Properties”, §1.6 “Simple Forms of Feedback”, §2.1 “Nonlinear Static Models”, §2.3 “Using Feedback to Attenuate Disturbances”, §2.4 “Using Feedback to Track Reference Signals”, §11.1 “Basic Control Functions”. Free: https://fbswiki.org (FBS)
- Lynch, K. M. and Park, F. C. (2017). *Modern Robotics: Mechanics, Planning, and Control*. Cambridge University Press. §8.1.1 Newton’s second law, §11.3 motion control with velocity inputs. Free preprint: http://modernrobotics.org (MR)
- OpenStax, *University Physics Volume 1*, §5.6 “Common Forces” (the weight’s component along a slope, w sin $\theta$), https://openstax.org/books/university-physics-volume-1/pages/5-6-common-forces (OpenStax §5.6)


**Other references**


- Northwestern Robotics, “Modern Robotics, Chapter 11.1: Control System Overview”, narration page, https://modernrobotics.northwestern.edu/nu-gm-book-resource/11-1-control-system-overview/ (a robot’s control loop of amplifiers, motors and encoders)
- Autoware Foundation, autoware_universe, the PID longitudinal controller: its README, its main source file and its parameter file (the controller’s folder under control/), https://github.com/autowarefoundation/autoware_universe (read for the code names and gains quoted in Section 7)
- ros-controls, ros2_controllers, the PID controller’s user documentation (file pid_controller/doc/userdoc.rst), https://github.com/ros-controls/ros2_controllers (read for the reference, state and command interfaces)


## 10. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

| Term | Meaning |
|---|---|
| Open-loop control (G-2940) | Choosing the commands from the plan and a model alone, with no measurement of what the system does, such as pushing a fixed 1 N because a test showed it gives 0.5 m/s; it is simple, but it cannot notice a ramp or a wrong model. |
| Control error (G-2943) | The reference minus the measured output, with its sign, such as $0.5 - 0.45 = 0.05$ m/s; a feedback controller computes its command from it, so a correction appears only once there is an error. |
| Feedback (G-2941) | Using a measurement of what a system does to decide its next command, such as pushing harder when the measured speed is too low; it lets a controller correct errors whatever caused them. |
| Closed-loop control (G-2942) | Control that uses feedback: the output is measured and compared with the reference, so the command depends on what the system really does; it holds a robot near its wanted speed on a ramp where open-loop control fails. |
| Block diagram (G-2950) | A picture of a control system with each part drawn as a box and each signal as an arrow between them, with circles where signals add or subtract; it shows where every signal, disturbance and noise enters the loop. |
| Summing junction (G-2951) | A circle in a block diagram where the arriving signals are added or subtracted, each with the sign written beside it, such as reference minus measured output giving the error. |
| Reference (G-2944) | The value we want the controlled output to have, such as a wanted speed of 0.5 m/s from the plan; the controller compares the measured output with it. |
| Controller (G-2947) | The part of a feedback loop that turns the control error into a command, such as a computer running “push 1 N plus 20 N per m/s of error”; it decides how hard to correct. |

| Term | Meaning |
|---|---|
| Actuator (G-2948) | The part of a feedback loop that turns the controller's command into a physical action on the plant, such as motors producing a force of at most 4 N; its limits bound what the controller can do. |
| Plant (G-2945) | The system being controlled, such as the robot's body rolling with its mass and drag; its output is the quantity we want to control, and actuator, plant and sensor are often lumped together under this name. |
| Sensor (control loop) (G-2946) | The part of a feedback loop that measures the plant's output, such as wheel encoders measuring speed; its reading is what the controller compares with the reference. |
| Disturbance (G-2949) | An unwanted outside effect that changes what the plant does, such as a ramp pulling a robot back with 0.785 N; it changes the true output, unlike sensor noise, which changes only the reading. |
| Regulation (G-2952) | Keeping an output close to a fixed reference in spite of disturbances, such as holding 0.5 m/s on a ramp; the controller's job is to undo whatever pushes the output away. |
| Setpoint (G-2953) | A reference that stays constant, such as a cruise speed of 0.5 m/s or a docking position; holding a setpoint is regulation. |
| Tracking (G-2954) | Making an output follow a reference that changes over time, such as a planned speed rising at 0.25 $\mathrm{m/s^2}$; also called the servo problem, it needs the controller to keep up with the change, and a correction from the error alone lags behind. |
| Newton's second law (G-2809) | The net force on a body equals its mass times its acceleration; it gives the force needed to change an object's speed, which the quasistatic assumption neglects. |
| Control input $u$ (G-2298) | The numbers we choose to drive a robot, such as $u = (v, \omega)$; the model turns them into motion. |
| Dot notation $\dot{x}$ (G-2296) | A dot over a quantity means its rate of change per second (its derivative with respect to time), such as $\dot{x} = 0.433$ m/s; models of motion are written with it. |

| Term | Meaning |
|---|---|
| State $x_t$ (G-2314) | The quantities a robot keeps track of at time $t$ because they decide what happens next, such as its pose, its speeds and moving things around it; it is what localization and filtering estimate. |
| Euler step (G-2299) | One step of predicting motion: hold the rates fixed for a short time $\Delta t$ and add rate times $\Delta t$ to each quantity; repeated, it traces a path, more accurately the smaller $\Delta t$ is. |
| Sensor noise (G-2312) | Small random errors in a sensor's readings, so the same quantity measured twice gives slightly different numbers (a wall 2 m away reads 1.92 to 2.07 m); it is one reason a robot cannot take a reading at face value. |